

Full Stack Application – Part 1



03

Web applications have become essential in modern life, transforming the way we access services, automate tasks, and interact seamlessly across various domains.

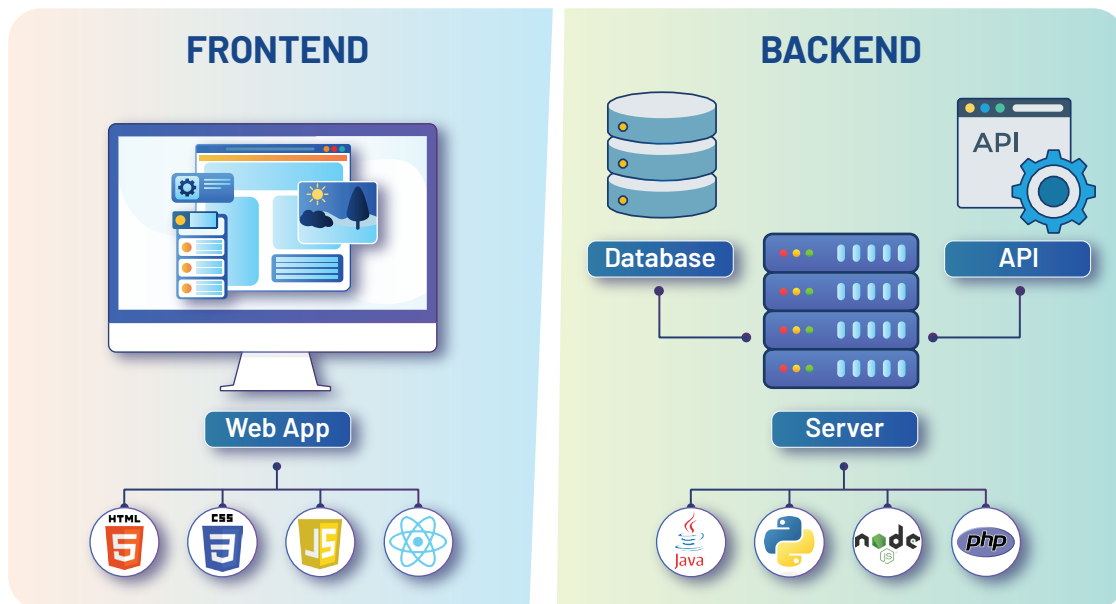


The digital era has integrated online processes into daily life, from shopping and food delivery to e-learning. Users can browse products, register for courses, and access information effortlessly. Everything updates instantly, and records are always accessible, ensuring a seamless experience. But have you ever considered the technology that enables this efficiency? This is where **Full Stack Development** comes into play!

In order to understand how the Full Stack applications function efficiently, we need to explore their two key components: **Frontend** and **Backend**.

Frontend vs Backend

Frontend: It is part of a website or web application that users see and interact with. Everything visible on a website, including text, images, buttons, and layouts, is part of the frontend. Technologies such as HTML, CSS, and JavaScript are used to build the frontend, often supported by frameworks like React, Angular, etc. to enhance functionality and user experience.



Backend: It is part of a web application responsible for processing and handling data. Operations such as fetching data from a database, storing information, and managing user requests are all part of the backend. Languages like PHP, Java, Python, and JavaScript are commonly used, along with frameworks such as Node.js, Django, Laravel, etc., to enhance functionality.

To summarise, the Frontend displays content and collects user input, while the Backend processes this input and stores it in a database.

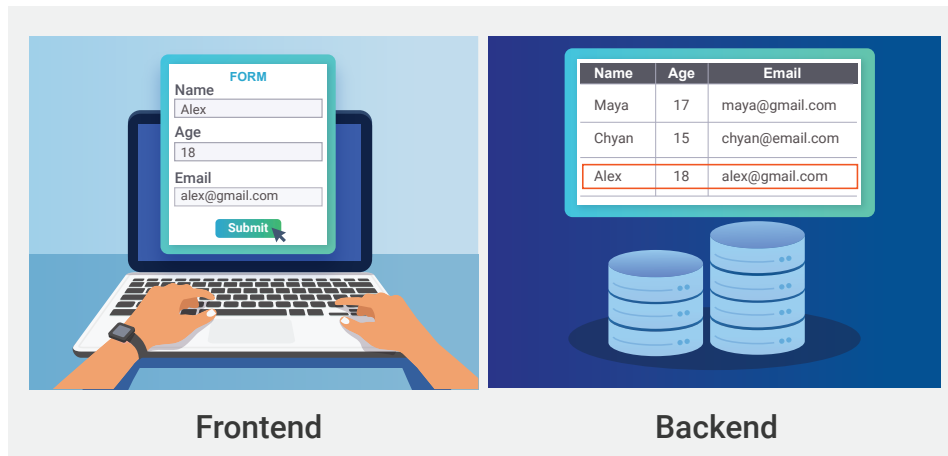
What is Full Stack Development?

Full Stack Development encompasses the entire process of application development, including both the frontend and backend. An individual who works on both aspects and can develop a complete web application is known as a Full Stack Developer.

Course Enrollment Application

In this lesson, we will build a Full Stack Course Enrollment Application with a minimalistic, interactive, and user-friendly interface. Users can insert and view data effortlessly, with properly labelled navigation links for a seamless experience.

We will build the frontend using React, allowing users to view student, course, and enrollments details. It will include forms for adding new students and enrollments, along with options to view stored records.



For the backend, we will use a MySQL database to store student, course, and enrollments details. Node.js will act as a bridge between the React frontend and the MySQL database, ensuring that data is stored correctly.

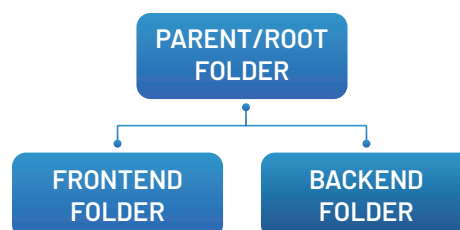
To keep the application simple, features like admin registration, login, logout, update, and delete are omitted. Once students grasp the basic functionalities, these can be added later.

With the basic understanding, the next step is to organise the application's files efficiently. Let us explore how a full-stack application's directory is structured.

Application's Folder/Directory Structure

Maintaining a well-organised folder structure is essential when developing a full-stack application. A proper folder structure ensures clarity, scalability, and ease of development.

Every full-stack application has a **root** (parent) folder, which contains two main folders—one for the frontend and the other for the backend.



The React project will be created inside the 'Frontend' folder, and the Node.js project will be created inside the 'Backend' folder.

To simplify learning, the application development process is split into two lessons. This lesson covers the frontend, while the next will focus on backend development. This step-by-step approach ensures a clear understanding of each part. Now, with this basic understanding, let us start building the application.

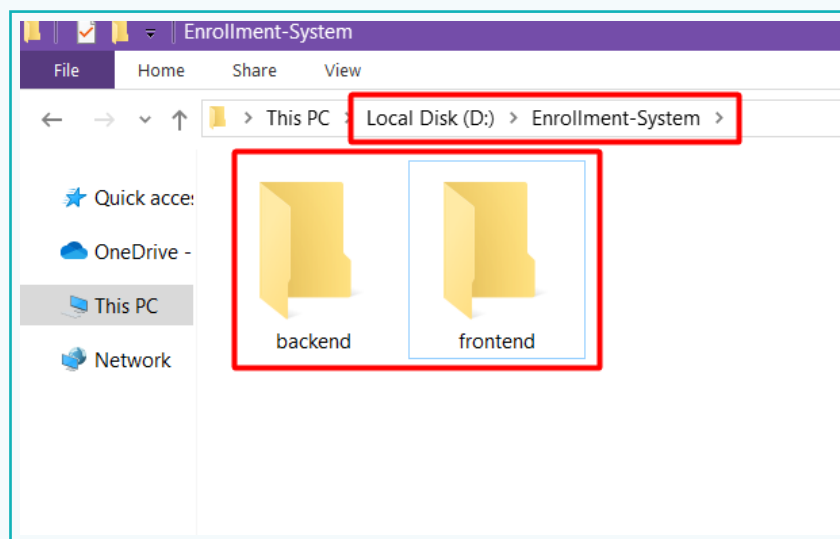
Activity

Build the frontend for the Full Stack Course Enrollment Application by creating a React project and designing the user interface.

Part 1 - Create a React Project

We will start by creating a root folder based on the directory structure.

- 1 Create a root folder and name it 'Enrollment-System'. Within this folder, create two additional folders, 'frontend' and 'backend'.

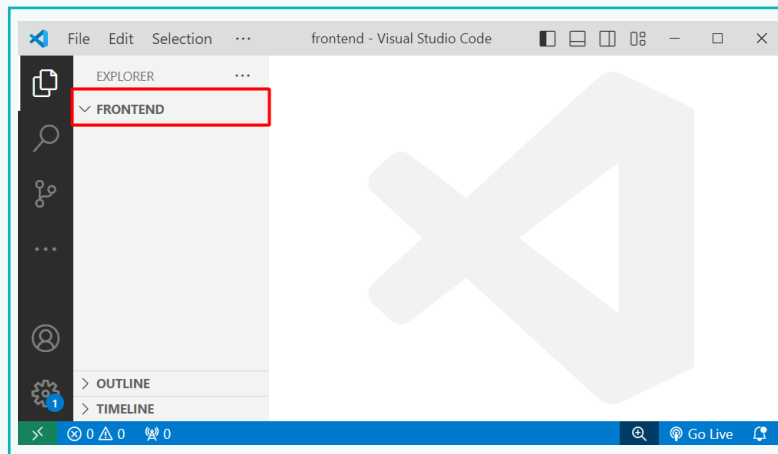


Note:

Ensure that the React project's folder name is in lowercase, as it does not support uppercase folder names.

Activity

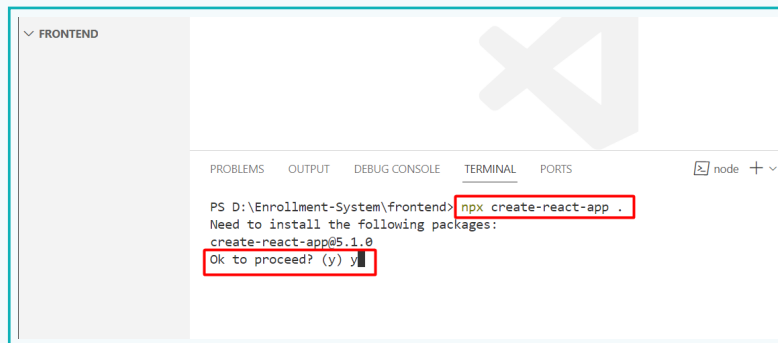
- 2 Since we will be creating only the frontend in this lesson, open the 'frontend' folder in VS Code.



Recall:

React is one of the most widely used component-based JavaScript libraries, used to develop the frontend of a web application.

- 3 Open the terminal in 'VS Code' and create a new React project using the command `"npx create-react-app"`. Type 'y' and press 'Enter' when asked to proceed.

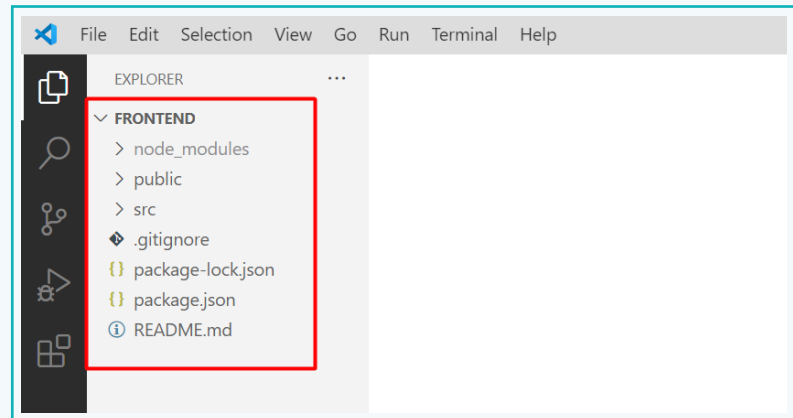


Note:

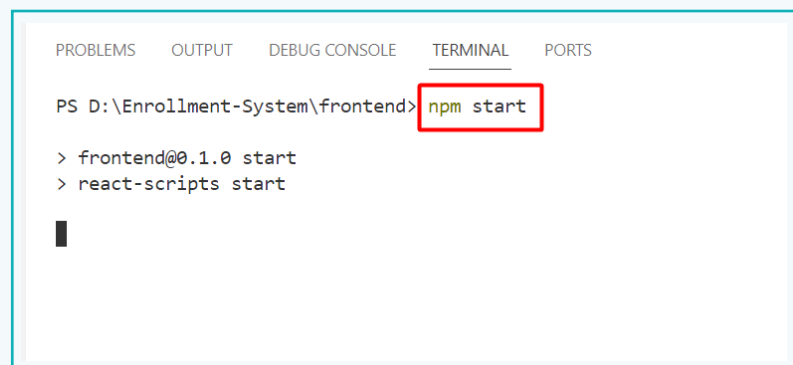
The 'dot' at the end of the command indicates that the React project should be created inside the same folder. If you want a different folder, simply replace the dot with the folder name.

Activity

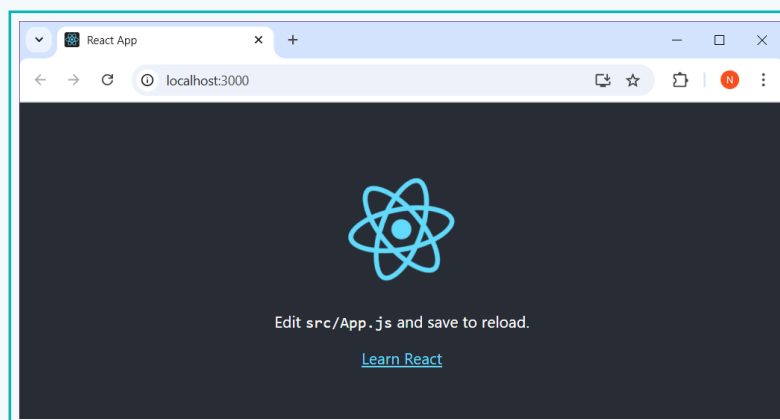
- 4 Once the project is created, the 'frontend' folder will contain multiple files and subfolders generated during the installation process.



- 5 Execute your project using the *npm start* command on the terminal.



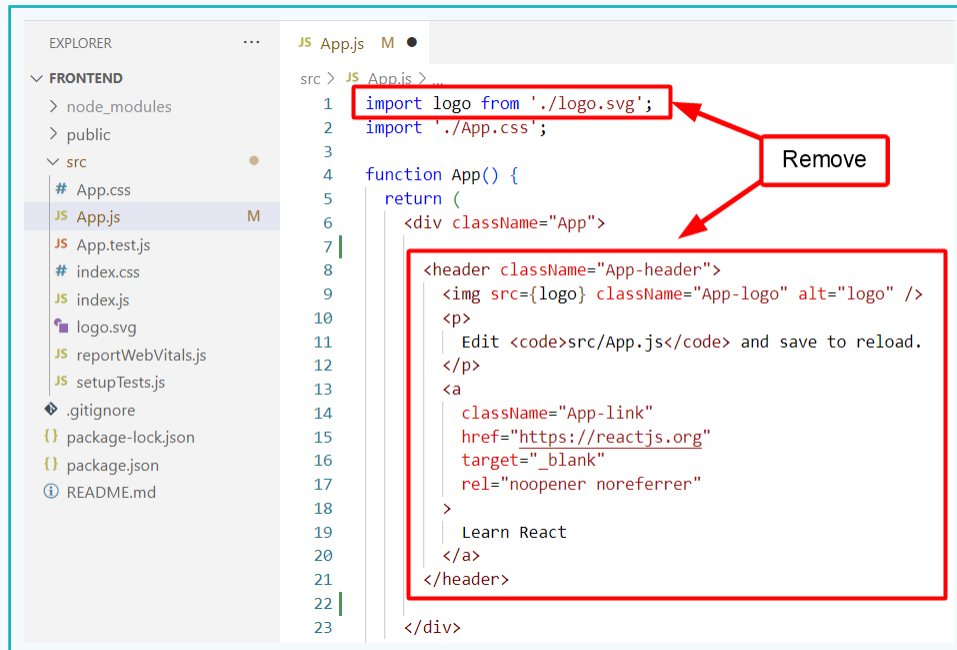
- A browser window will be launched displaying the output.



This is the default output of a React project.

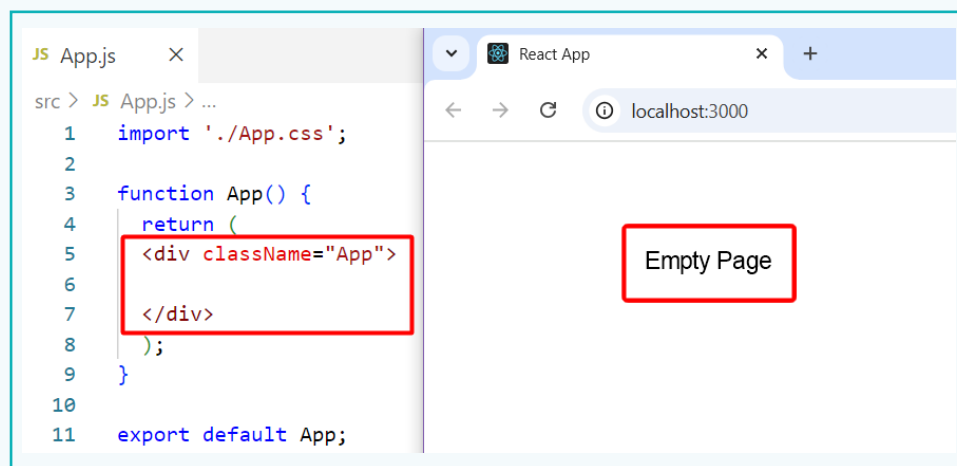
Activity

- 6 Since we will be adding our own content, we will remove the default output. This can be done by editing the 'App.js' file located in the 'src' folder.



```
1 import logo from './logo.svg';
2 import './App.css';
3
4 function App() {
5   return (
6     <div className="App">
7
8       <header className="App-header">
9         <img src={logo} className="App-logo" alt="logo" />
10        <p>
11          Edit <code>src/App.js</code> and save to reload.
12        </p>
13        <a
14          className="App-link"
15          href="https://reactjs.org"
16          target="_blank"
17          rel="noopener noreferrer"
18        >
19          Learn React
20        </a>
21      </header>
22    </div>
23  )
24}
```

- 7 We now have an empty page as the output.



```
1 import './App.css';
2
3 function App() {
4   return (
5     <div className="App">
6
7     </div>
8   );
9 }
10
11 export default App;
```

Next, we will create components for our webpage.

Activity

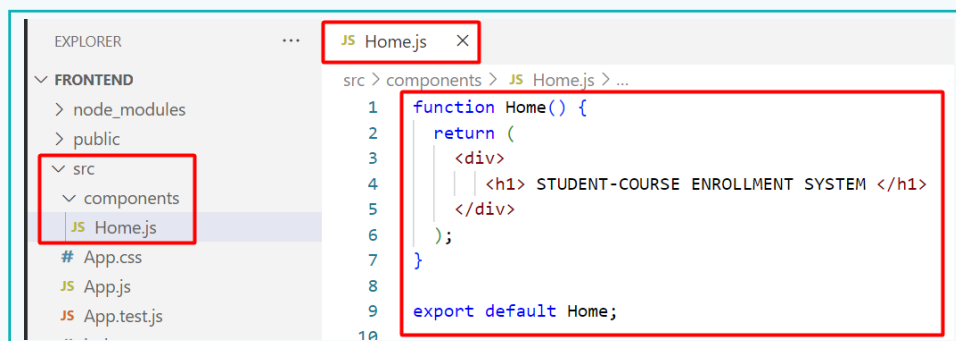
Part 2 - Create Home Component

Since React follows a component-based design approach, the frontend will be built using various modular and reusable components. In our application, the user interface can be structured into the following key components:

- **Home.js:** Homepage for the application.
- **Sidebar.js:** A navigation panel component that provides quick access to different sections, such as Courses, Students, and Enrollments.
- **StudentForm.js:** A form component to collect student details.
- **StudentProfile.js:** A component to display the stored student details.
- **Courses.js:** A component to display all the courses.

As multiple components need to be created, it is recommended that they be placed in a separate folder. This keeps your components organised and easy to manage.

- 1 Create a folder named 'components' within the 'src' directory. Inside this folder, create our first component, 'Home.js'.
- 2 Add the heading 'STUDENT-COURSE ENROLLMENT SYSTEM' in this component.



The screenshot shows the VS Code interface. On the left, the 'EXPLORER' sidebar displays the project structure: 'FRONTEND' > 'node_modules' > 'public' > 'src' > 'components' > 'JS Home.js'. The 'components' folder and 'JS Home.js' file are highlighted with red boxes. On the right, the 'JS Home.js' file is open, showing the following code:

```
1 function Home() {  
2   return (  
3     <div>  
4       <h1> STUDENT-COURSE ENROLLMENT SYSTEM </h1>  
5     </div>  
6   );  
7 }  
8  
9 export default Home;  
10
```

Recall:

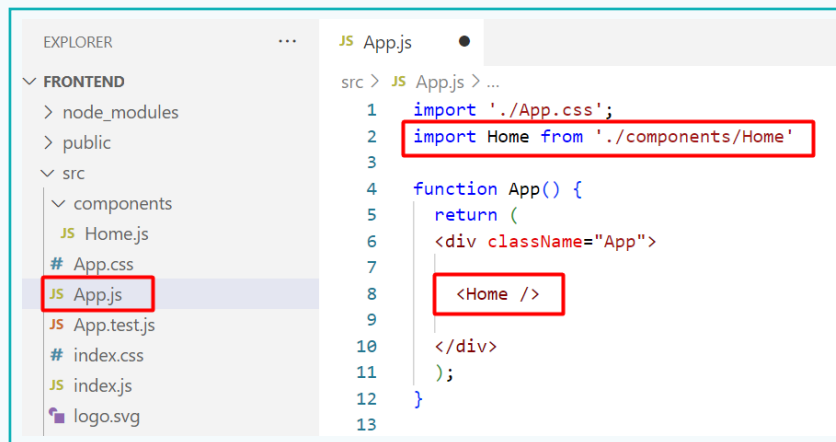
Every React component file has a fixed format consisting of:

- A **function** and a **return** statement, where we define our component and add content to it.
- An **export** statement to make the component available for use by other components within the application.

Activity

- 3 Navigate to the root component 'App.js' file and import the 'Home' component.
 - Next, declare the component within the return statement to make it appear on the output screen.

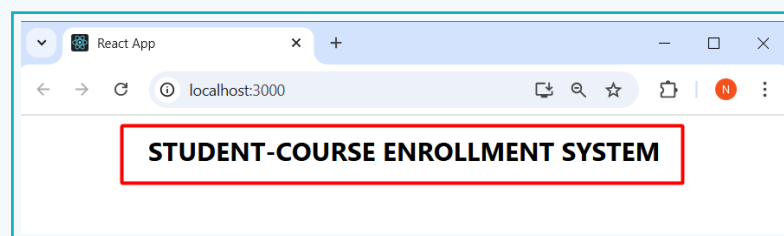
Recall that, to display a user-created component, we must declare it in the 'App.js' file.



```
EXPLORER
  FRONTEND
    node_modules
    public
    src
      components
        Home.js
      App.css
      App.js
      App.test.js
      index.css
      index.js
      logo.svg

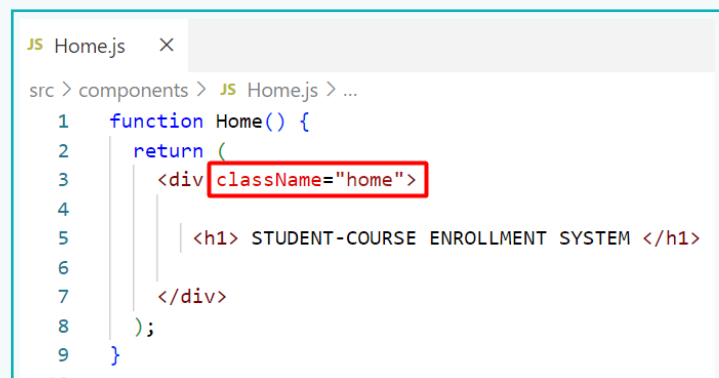
src > JS App.js > ...
1  import './App.css';
2  import Home from './components/Home'
3
4  function App() {
5    return (
6      <div className="App">
7
8        <Home />
9
10     </div>
11   );
12 }
13
```

- Execute the application. It will display the output 'STUDENT-COURSE ENROLLMENT SYSTEM'.



We will also design the component using some basic CSS.

- 4 Open the 'Home.js' file and assign the class *home* to the <div> using the *className* property.



```
JS Home.js
src > components > JS Home.js > ...
1  function Home() {
2    return (
3      <div className="home">
4
5        <h1> STUDENT-COURSE ENROLLMENT SYSTEM </h1>
6
7      </div>
8    );
9  }
10
```

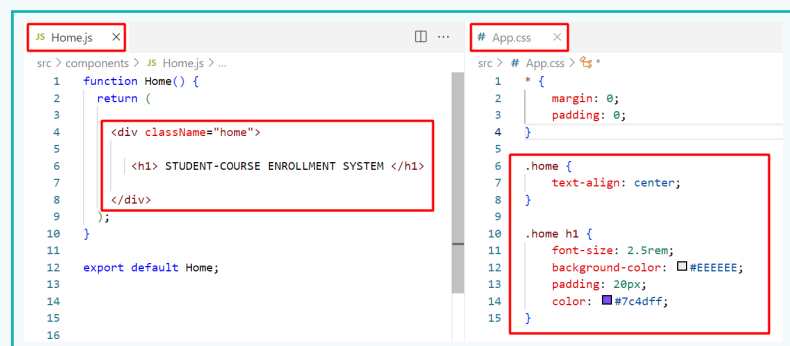
Activity

Note:

React already provides the 'App.css' file for styling purposes. Open the file and remove any existing CSS properties, as we will add our own CSS design properties.

- 5 Open the 'App.css' file and add the following CSS properties for the 'Home' component.

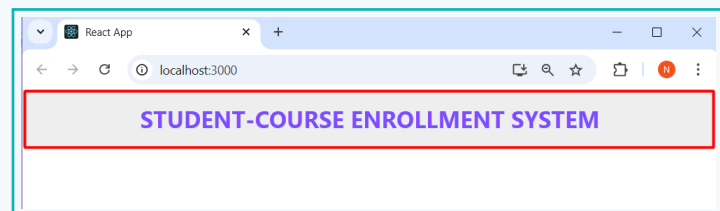
- Using the class *home*, refer to the `<div>` tag and the `<h1>` tag in the 'Home.js' file and apply CSS as shown.



```
src > components > JS Home.js > ...
1 function Home() {
2   return (
3     <div className="home">
4       <h1> STUDENT-COURSE ENROLLMENT SYSTEM </h1>
5     </div>
6   );
7 }
8
9 export default Home;
10
11
12
13
14
15
16
```

```
src > # App.css > *
1 * {
2   margin: 0;
3   padding: 0;
4 }
5
6 .home {
7   text-align: center;
8 }
9
10 .home h1 {
11   font-size: 2.5rem;
12   background-color: #EEEEEE;
13   padding: 20px;
14   color: #7c4dff;
15 }
```

- After adding CSS, execute the application and check the 'Home' component.

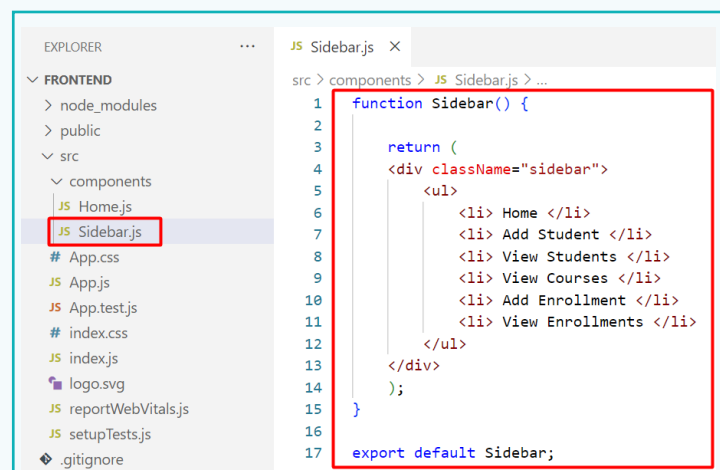


Next, we will create a sidebar for navigation. This component will contain links for navigation and provide users with quick access to different sections or pages in the application.

Part 3 - Create the Sidebar Component

- 1 Create a 'Sidebar.js' component.

- Use an 'unordered list (`<ul>`)' to create the list.



```
EXPLORER
...
FRONTEND
  > node_modules
  > public
  > src
    > components
      JS Home.js
      JS Sidebar.js
      # App.css
      JS App.js
      JS App.test.js
      # index.css
      JS index.js
      logo.svg
      JS reportWebVitals.js
      JS setupTests.js
      .gitignore

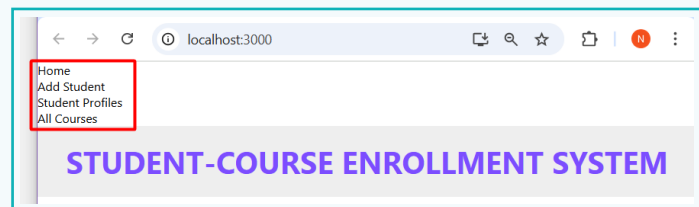
src > components > JS Sidebar.js > ...
1 function Sidebar() {
2
3   return (
4     <div className="sidebar">
5       <ul>
6         <li> Home </li>
7         <li> Add Student </li>
8         <li> View Students </li>
9         <li> View Courses </li>
10        <li> Add Enrollment </li>
11        <li> View Enrollments </li>
12      </ul>
13    </div>
14  );
15 }
16
17 export default Sidebar;
```

Activity

- 2 Import the 'Sidebar.js' component in the 'App.js' file and use it.

```
JS App.js x
src > JS App.js > ...
1  import './App.css';
2  import Home from './components/Home'
3  import Sidebar from './components/Sidebar'
4
5  function App() {
6    return (
7      <div className="App">
8        <Sidebar />
9        <Home />
10     </div>
11   );
12 }
```

- Execute the application after adding the Sidebar component.



Observe that the Sidebar and Home components are not properly aligned. To fix this, we will implement a column layout, positioning the components side by side instead of stacking them vertically.

Part 4 - Create a 2-Column Layout

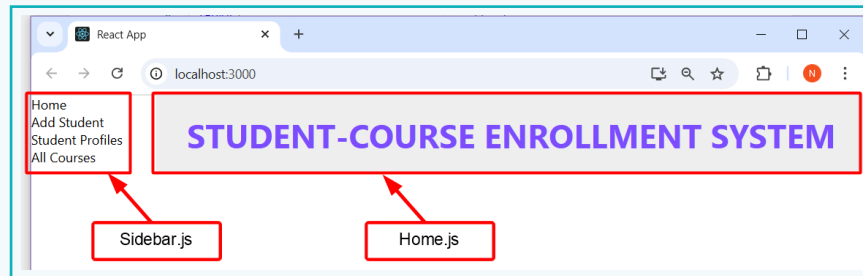
- 1 Open the 'App.css' file and adjust the layout using CSS Grid.
- We will convert the main container with class 'App' into a grid container using the *display: grid* property.
 - Next, use the *grid-template-columns* property to define a two-column layout, with the first and second columns having widths of 15% and 85%, respectively.

```
App.js
src > JS App.js > ...
1  import './App.css';
2  import Home from './components/Home'
3  import Sidebar from './components/Sidebar'
4
5  function App() {
6    return (
7      <div className="App">
8        <Sidebar />
9        <Home />
10     </div>
11   );
12 }
```

```
# App.css
src > # App.css > ...
10 .home h1 {
11   font-size: 2.5rem;
12   background-color: #EEEEEE;
13   padding: 20px;
14   color: #7c4dff;
15 }
16
17 .App {
18   display: grid;
19   grid-template-columns: 15% 85%;
20 }
21
```

Activity

- The 'Sidebar.js' component will now appear on the left side, with the 'Home.js' component occupying the rest of the page.

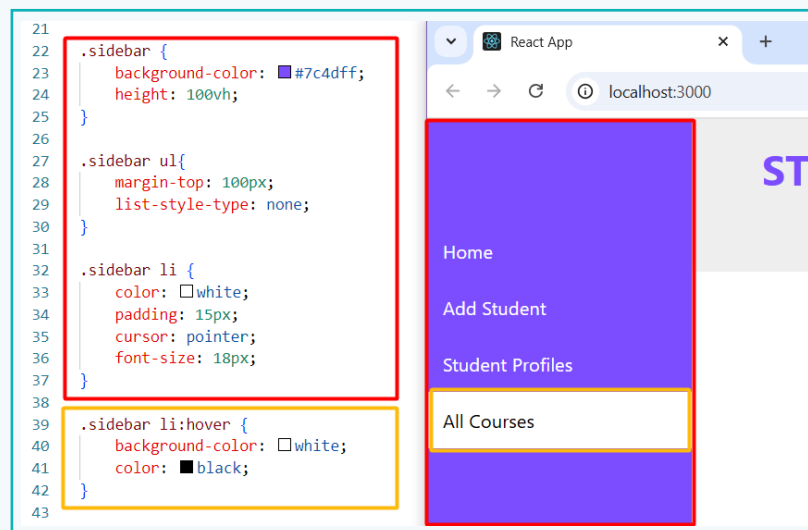


Observe that the Sidebar lacks styling. We will enhance its appearance by applying CSS.

Part 5 - Designing the Sidebar

- In the 'App.css' file, refer to the *sidebar* class and the unordered list, and apply the following CSS properties.

- We will also use *':hover'* to create an effect when users hover their mouse over the list items.



Next, we will create a component for the Student Form.

Activity

Part 6 - Create the 'StudentForm' Component

The 'StudentForm.js' React component will be used to enter and submit student-related details.

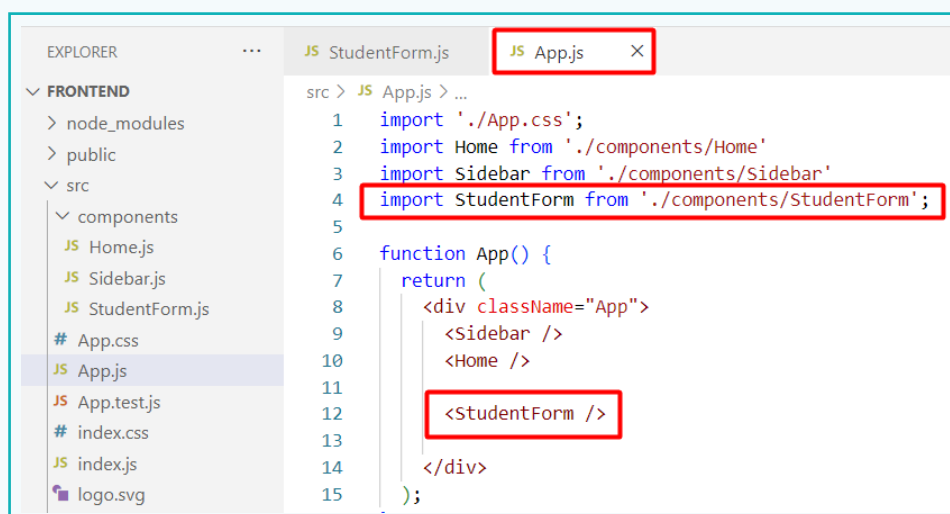
- 1 Create a 'StudentForm.js' component and add an HTML form with 'Name', 'Email', 'Grade', 'Course', and 'Submit button' fields.



The screenshot shows the VS Code interface. On the left, the Explorer pane shows the file structure with 'StudentForm.js' highlighted in the 'components' folder. On the right, the editor shows the code for 'StudentForm.js'.

```
1 function StudentForm() {  
2   return (  
3     <div className='student-form'>  
4       <h2> ADD STUDENT </h2>  
5       <form>  
6         <input type='text' placeholder='Enter Student Name' />  
7         <input type='text' placeholder='Enter Student Email' />  
8         <input type='text' placeholder='Enter Student Grade' />  
9         <select>  
10          <option value="" disabled selected> Select a Course </option>  
11          <option value="WEB-101"> Learning HTML and CSS </option>  
12          <option value="CS-102"> Starting with Computer Science </option>  
13          <option value="DS-103"> Data Science for Beginners </option>  
14        </select>  
15        <br />  
16        <button type='submit'> Submit </button>  
17      </form>  
18    </div>  
19  );  
20 }  
21 export default StudentForm;  
22
```

- 2 Import the 'StudentForm' component in the 'App.js' file and declare it within the *return* statement.

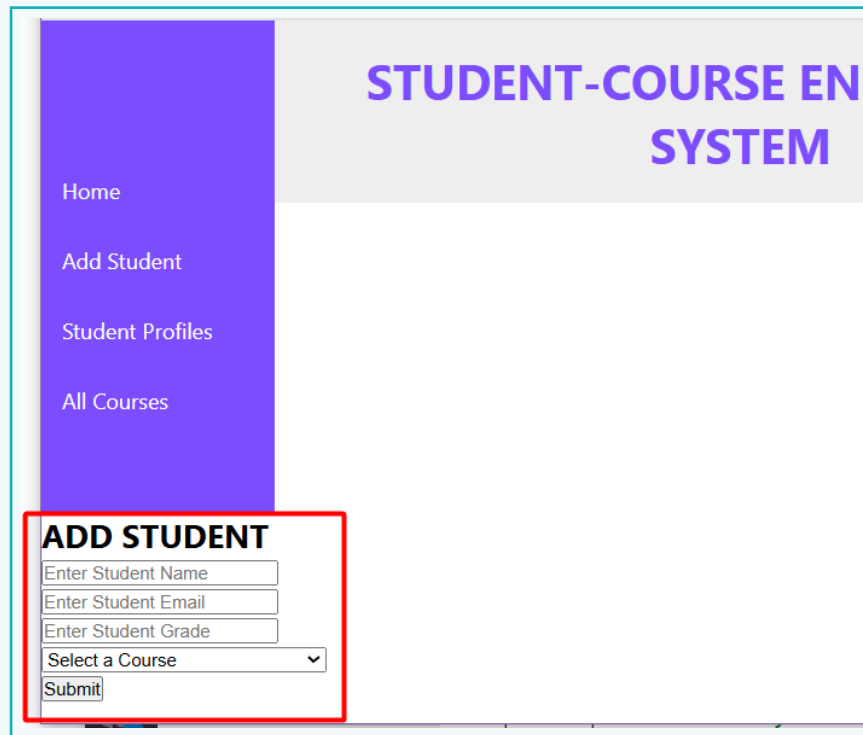


The screenshot shows the VS Code interface. On the left, the Explorer pane shows the file structure with 'App.js' highlighted in the 'components' folder. On the right, the editor shows the code for 'App.js'.

```
1 import './App.css';  
2 import Home from './components/Home'  
3 import Sidebar from './components/Sidebar'  
4 import StudentForm from './components/StudentForm';  
5  
6 function App() {  
7   return (  
8     <div className="App">  
9       <Sidebar />  
10      <Home />  
11      <StudentForm />  
12    </div>  
13  );  
14 }  
15
```

Activity

- Execute the application to check the output.



The screenshot shows a web application interface. On the left is a purple sidebar with the following links: Home, Add Student, Student Profiles, and All Courses. The main content area has a header that says "STUDENT-COURSE EN SYSTEM". Below the header, the "ADD STUDENT" form is visible, which is highlighted with a red rectangular box. This form includes the following elements: a text input field labeled "Enter Student Name", another text input field labeled "Enter Student Email", a third text input field labeled "Enter Student Grade", a dropdown menu labeled "Select a Course" with a downward arrow, and a "Submit" button at the bottom.

Observe that the 'StudentForm' component is displayed below the 'Home' and 'Sidebar' components in the output. This occurs because the 'StudentForm' component is placed directly beneath them in the 'App.js' file. As we add more components, they will appear sequentially as per their placement in the 'App.js' file.

Since this is a single-page application, this issue can be resolved by displaying only one component at a time instead of all components together. This can be achieved by defining routes using the **React Router** library.

Recall:

React Router is a library for handling navigation in React applications. It allows us to define dynamic routes and map components to specific URLs. It helps manage the UI and displays specific components instead of showing all the components together.

Activity

React Router Components

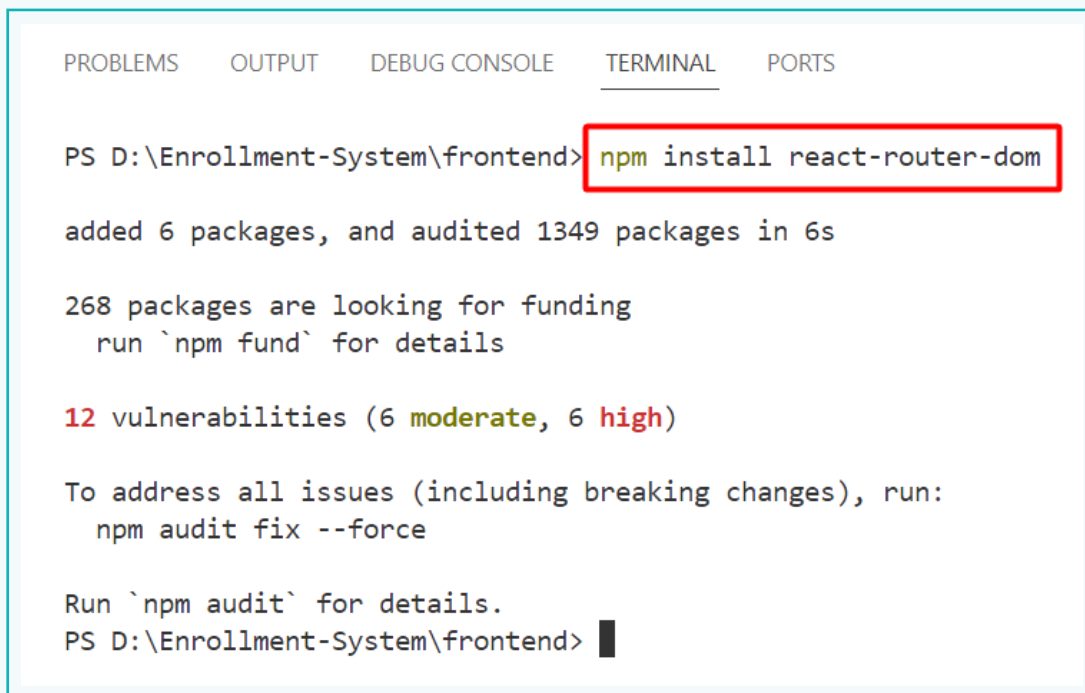
React Router provides the following components to help with navigation and defining routes.

- **<BrowserRouter>**: Wraps the entire application and enables routing functionality.
- **<Routes>**: Acts as a container for all the routes used in the application.
- **<Route>**: Specifies the component to be rendered for a given URL.
- **<Link>**: Creates a navigational link for your routes.

Part 7 - Define Routes using React Router

Install React Router

- 1 To install the React Router, open the 'Terminal' in VS Code and execute the command `npm install react-router-dom`. In case the React application is still executing, close it by pressing 'Ctrl + C' and then execute the installation command.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Enrollment-System\frontend> npm install react-router-dom

added 6 packages, and audited 1349 packages in 6s

268 packages are looking for funding
  run `npm fund` for details

12 vulnerabilities (6 moderate, 6 high)

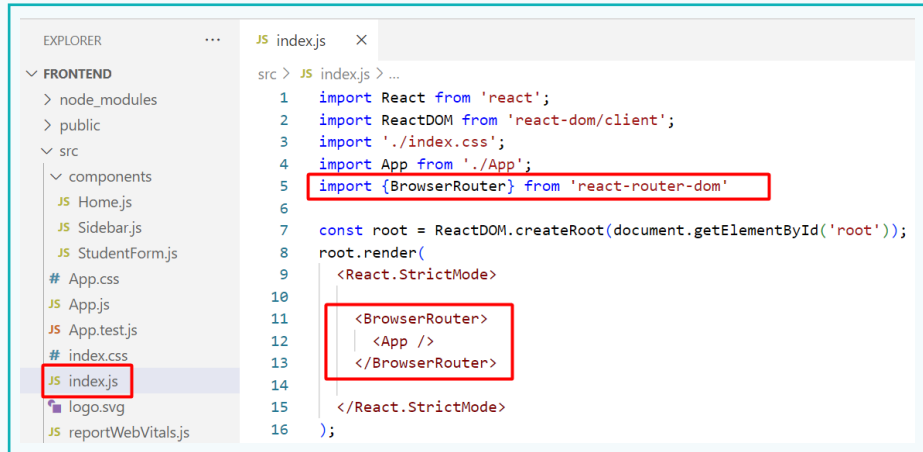
To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS D:\Enrollment-System\frontend> 
```

Now, we will create routes using the React Router library.

Activity

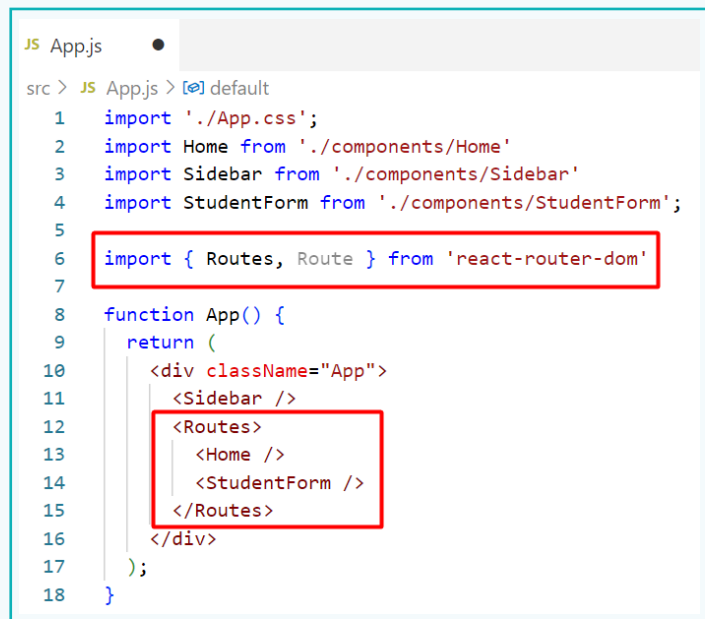
- 2 First, we need to open the 'index.js' file and wrap the root component `<App />` with the `<BrowserRouter>` component. This will enable routing throughout the application.



```
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import './index.css';
4 import App from './App';
5 import {BrowserRouter} from 'react-router-dom'
6
7 const root = ReactDOM.createRoot(document.getElementById('root'));
8 root.render(
9   <React.StrictMode>
10     <BrowserRouter>
11       <App />
12     </BrowserRouter>
13   </React.StrictMode>
14 );
```

Next, we will define the routes and map them with specific components.

- 3 In the 'App.js' file, import the `<Routes>` and `<Route>` components. Wrap the components ('Home.js' and 'StudentForm.js') that we created within the `<Routes>` component.



```
1 import './App.css';
2 import Home from './components/Home';
3 import Sidebar from './components/Sidebar';
4 import StudentForm from './components/StudentForm';
5
6 import { Routes, Route } from 'react-router-dom'
7
8 function App() {
9   return (
10     <div className="App">
11       <Sidebar />
12       <Routes>
13         <Home />
14         <StudentForm />
15       </Routes>
16     </div>
17   );
18 }
```

Note:

The 'Sidebar.js' component is not placed within `<Routes>` as it is meant to be displayed constantly. If you wrap 'Sidebar.js' inside `<Routes>`, it will only appear when there is a matching route.

Activity

4 Now, use the `<Route>` component to define which component to display when a user visits a specific route. This component has two properties:

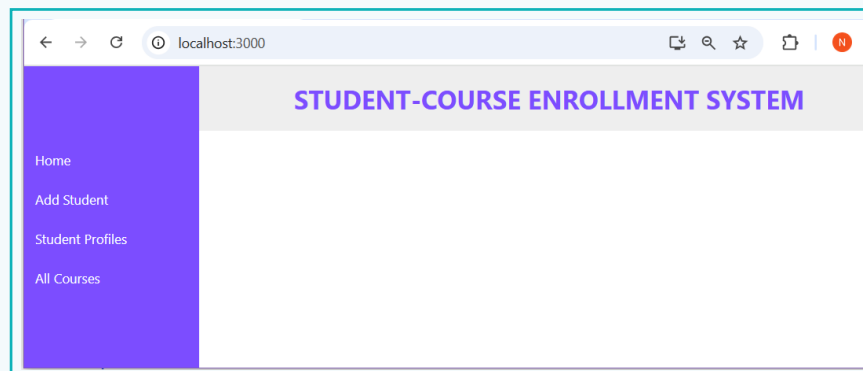
- *path*: Specifies the URL for this route.
- *element*: Specifies the component to be rendered.

```
src > JS App.js > App
1  import './App.css';
2  import Home from './components/Home'
3  import Sidebar from './components/Sidebar'
4  import StudentForm from './components/StudentForm';
5  import { Routes, Route } from 'react-router-dom'
6
7  function App() {
8    return (
9      <div className="App">
10        <Sidebar />
11        <Routes>
12          <Route path="/" element={<Home />} />
13          <Route path="/add-student" element={<StudentForm />} />
14        </Routes>
15      </div>
16    )
17  }
```

Note:

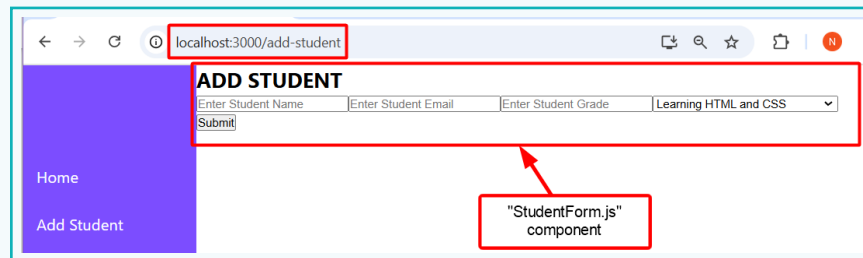
“/” refers to the root URL, which in our case is ‘localhost:3000’ and ‘/add-student’ refers to the ‘localhost:3000/add-student’ URL.

- If you check the output now, the ‘StudentForm’ component will not be displayed.



Activity

- To display the 'StudentForm.js' component, we need to visit the URL 'localhost:3000/add-student', as this was the route we had specified in the *path* property in the 'App.js' file.



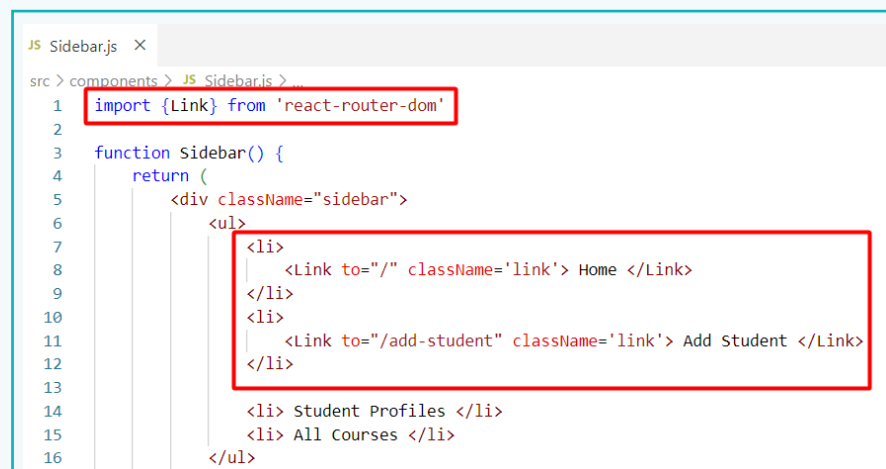
Navigation is an essential part of any web application, allowing users to move between different sections seamlessly. Next, we will implement navigation using some functionalities from the React Router library.

Part 8 - Link Routes with the Sidebar Items

To simplify navigation, we will link the routes with list items in the Sidebar. This will enable users to access the desired components simply by clicking the appropriate items on the Sidebar. To achieve this functionality, we will use the `<Link>` component from React Router.

- 1 Import the `<Link>` component and wrap the 'Home' and 'Add Student' list items within the `<Link>` component. This component uses the *to* property, where we can specify the appropriate routes, as defined in the 'App.js' file.

- Add a class *link* as well, which we can use to design the links.



Try clicking on the 'Home' or 'Add Student' link in the Sidebar and see how the application renders the associated components.

Activity

- 2 Open the 'App.css' file and add the following CSS properties to design the list items.

```

39 .sidebar li:hover {
40   background-color: white;
41   color: black;
42 }
43
44 .sidebar li:hover .link{
45   color: black;
46 }
47
48 .link {
49   text-decoration: none;
50   color: white;
51 }
52

```

Next, we will design the student form.

Part 9 - Design the StudentForm component

- 1 Add the following CSS properties in the 'App.css' file to design the form.

```

54 .student-form {
55   margin-top: 100px;
56   text-align: center;
57 }
58
59 .student-form h2{
60   font-size: 2.5rem;
61   color: #7c4dff;
62   margin-bottom: 20px;
63 }
64
65 input[type='text'], select {
66   border: 1px solid grey;
67   padding: 15px 10px;
68   width: 40vw;
69   margin-bottom: 20px;
70   border-radius: 5px;
71 }
72

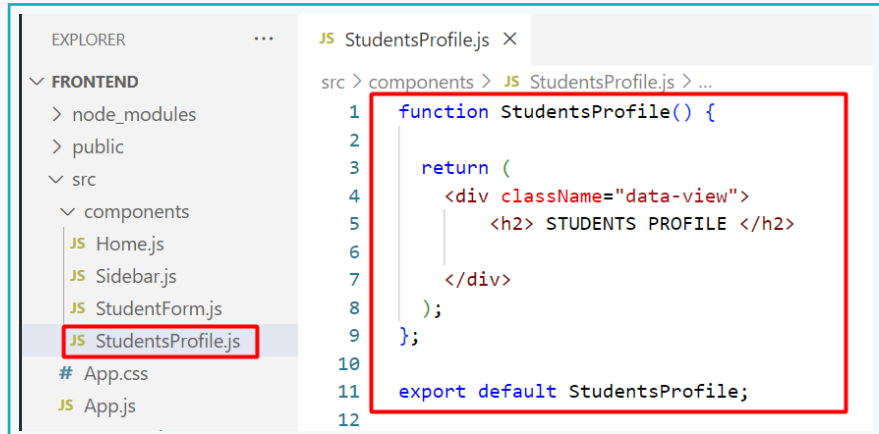
```

Next, we will create a component to display all the student records.

Activity

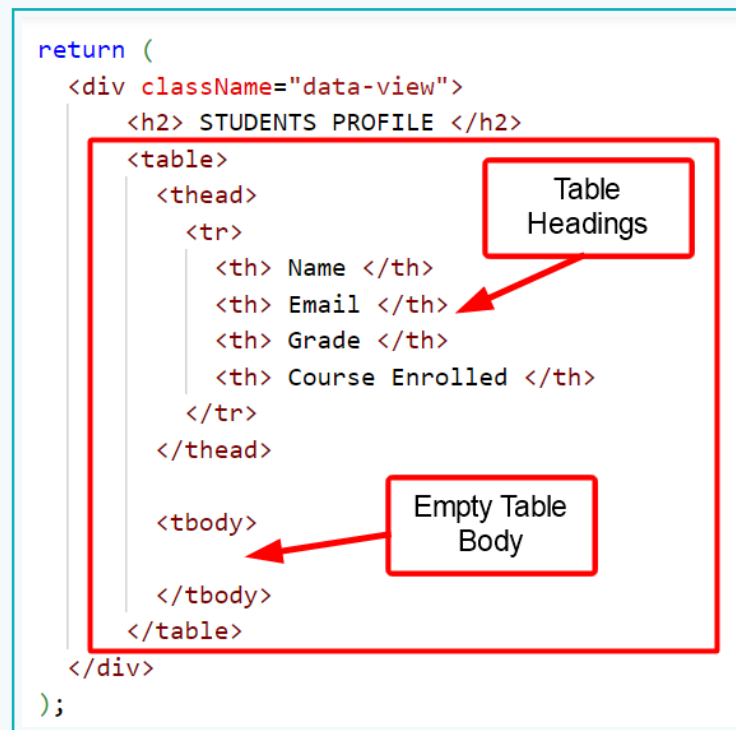
Part 10 - Create and Design the StudentProfile Component

- 1 Create a 'StudentsProfile.js' component. Assign a class 'data-view' to the `<div>` tag and add the heading "Students Profile".



```
1 function StudentsProfile() {
2
3   return (
4     <div className="data-view">
5       <h2> STUDENTS PROFILE </h2>
6     </div>
7   );
8 }
9
10 export default StudentsProfile;
```

- 2 Create a table using the `<table>` tag and add column headings for Student's 'Name, Email, Grade and Course'. Leave the table body (`<tbody>`) tag empty, we will fill it later with data from the backend.



```
return (
  <div className="data-view">
    <h2> STUDENTS PROFILE </h2>
    <table>
      <thead>
        <tr>
          <th> Name </th>
          <th> Email </th>
          <th> Grade </th>
          <th> Course Enrolled </th>
        </tr>
      </thead>
      <tbody>
      </tbody>
    </table>
  </div>
);
```

Activity

- 3 Import the component in the 'App.js' file and define a route for it.

```
JS App.js
src > JS App.js > ...
5 import { Routes, Route } from 'react-router-dom'
6 import StudentsProfile from './components/StudentsProfile';
7
8 function App() {
9   return (
10     <div className="App">
11       <Sidebar />
12       <Routes>
13         <Route path="/" element={<Home />} />
14         <Route path="/add-student" element={<StudentForm />} />
15         <Route path="/students-profile" element={<StudentsProfile />} />
16       </Routes>
17     </div>
18   )
19 }
```

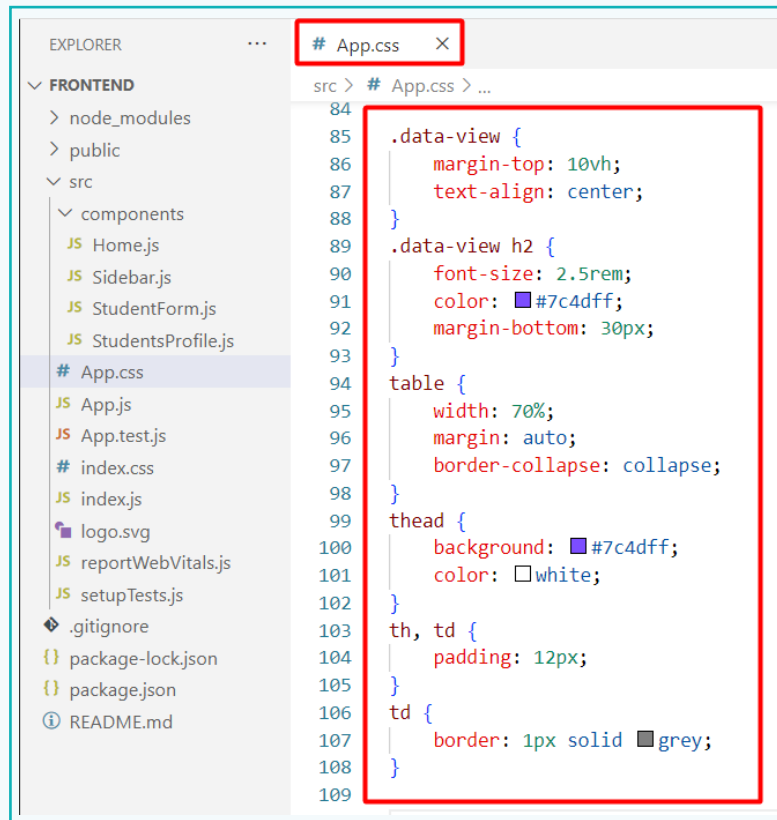
- 4 Link the route '/students-profile' to the 'Student Profiles' item in the 'Sidebar.js' file.

```
JS Sidebar.js X
src > components > JS Sidebar.js > [0] default
1 import {Link} from 'react-router-dom'
2 function Sidebar() {
3   return (
4     <div className="sidebar">
5       <ul>
6         <li>
7           <Link to="/" className='link'> Home </Link>
8         </li>
9         <li>
10          <Link to="/add-student" className='link'> Add Student </Link>
11        </li>
12        <li>
13          <Link to="/students-profile" className='link'> Student Profiles </Link>
14        </li>
15        <li> All Courses </li>
16      </ul>
17    </div>
18  )
19 }
```

Once done, try clicking on 'Student Profiles' in the sidebar. It should now display the 'StudentsProfile' component.

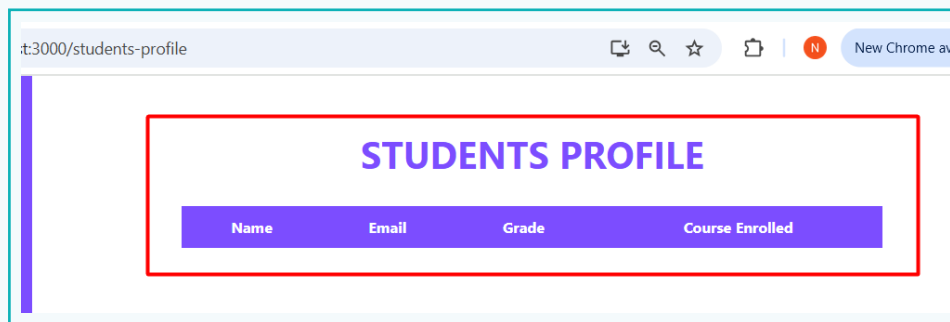
Activity

- 5 Open the 'App.css' file and apply CSS styling to enhance the component's appearance.



```
84
85 .data-view {
86   margin-top: 10vh;
87   text-align: center;
88 }
89 .data-view h2 {
90   font-size: 2.5rem;
91   color: #7c4dff;
92   margin-bottom: 30px;
93 }
94 table {
95   width: 70%;
96   margin: auto;
97   border-collapse: collapse;
98 }
99 thead {
100   background: #7c4dff;
101   color: white;
102 }
103 th, td {
104   padding: 12px;
105 }
106 td {
107   border: 1px solid grey;
108 }
109
```

- Run the application and observe the component's appearance after applying the specified CSS. Ensure that the styling has been applied correctly, and the layout is as expected.



The UI of our Course Enrollment Application is ready.

In this lesson, we built the frontend of a Full Stack Course Enrollment Application using React. We structured the project, created key components, applied CSS for styling, and used React Router for navigation. This setup ensures an interactive and well-organised interface.

In the next lesson, we will develop the backend using MySQL and Node.js to handle data processing and storage.

Exercise

Choose the Correct Answer.

- 1 Which of the following is used in frontend development?
 - a. MySQL
 - b. Node.js
 - c. React
 - d. Express

- 2 Which of the following statements is correct about React?
 - a. It is a JavaScript library
 - b. It is an HTML/CSS library
 - c. It is used for creating databases
 - d. It is a programming language

- 3 Which of the following is the root component file?
 - a. App.css
 - b. App.js
 - c. Home.js
 - d. package.json

- 4 Which of the following components does not belong to the React Router library?
 - a. <Routes>
 - b. <Link>
 - c. <App>
 - d. <BrowserRouter>

Exercise

Fill in the Blanks.

- 5 To run a React project, the _____ command is used.
- 6 To create a grid layout, we need to set the CSS *display* property to _____.
- 7 The 'element' property belongs to the _____ component.

Tech Flash

- | | |
|---------------------|---|
| React | : A JavaScript library to design the user interface of a website or a web application. |
| CSS Grid | : A layout system in CSS that allows you to create complex and responsive web layouts easily by defining a grid structure made up of rows and columns. |
| HTML Forms | : Web elements that are used to collect user input and send it to a server for processing. Forms are essential for things like login pages, search boxes, contact forms, and order forms. |
| React Router | : A standard library for client-side routing in React applications. |